Operating Systems: File and Memory Management

Q3. Summarize on Thrashing.

Definition: Thrashing occurs in a virtual memory system when a process spends more time paging (swapping pages in and out of memory) than actually executing. This usually happens when the degree of multiprogramming is too high, leading to a shortage of frames.

Mechanism:

- If a process does not have enough pages (frames) to support its active working set, it will immediately page fault.
- To bring in the required page, it must replace an existing page. However, since all pages are in active use, the replaced page is needed again almost immediately.
- This results in a continuous stream of page faults.
- Consequently, CPU utilization drops drastically. The operating system, seeing low CPU utilization, might attempt to increase the degree of multiprogramming (admitting more processes), which only worsens the situation.

Prevention:

- **Working Set Model:** Ensure the system allocates enough frames to a process to cover its current locality of reference.
- **Page Fault Frequency (PFF):** Monitor the page-fault rate; if it is too high, allocate more frames; if too low, reduce the frames.

Q4. What is a file? What are its attributes? Explain file operations.

1. Definition of a File

A file is a named collection of related information that is recorded on secondary storage. From a user's perspective, a file is the smallest allotment of logical secondary storage; data cannot be written to secondary storage unless it is within a file.

2. File Attributes

File attributes vary by operating system but typically include:

- **Name:** The symbolic name used by humans.
- **Identifier:** A unique tag (usually a number) used by the file system.
- **Type:** Needed for systems that support different types (e.g., .txt, .exe).
- **Location:** A pointer to a device and to the location of the file on that device.
- **Size:** The current size of the file (in bytes, words, or blocks).
- **Protection:** Access-control information determines who can read, write, or execute.
- **Time, date, and user identification:** Data for protection, security, and usage monitoring.

3. File Operations

The operating system provides system calls to create, manage, and delete files:

- **Creating a file:** Two steps are necessary: finding space in the file system and making an entry in the directory.
- **Writing a file:** The OS looks up the location and maintains a write pointer to the next write location.
- **Reading a file:** The OS maintains a read pointer to the location in the file where the next read is to take place.
- **Repositioning within a file (Seek):** The current file-position pointer is repositioned to a given value. No I/O is usually needed.
- **Deleting a file:** Release the file space and remove the directory entry.
- **Truncating a file:** The user erases the contents of a file but keeps its attributes.

Q5. With neat diagram explain different access methods.

Files store information, and this information must be accessed and read into computer memory.

1. Sequential Access

This is the most common method. Information in the file is processed in order, one record after the other. This mode is common for compilers and editors.

- **Operations:** Read next, Write next, Rewind.

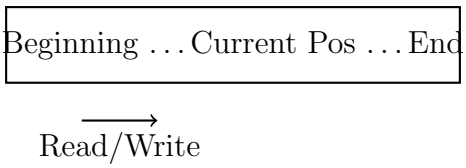

Figure 1: Sequential Access Model

2. Direct (Random) Access

A file is made up of fixed-length logical records that allow programs to read and write records rapidly in no particular order. This is based on the disk model of a file.

- **Operations:** Read block n, Write block n.
- Useful for databases where immediate access to large amounts of information is required.

3. Indexed Access

This method involves constructing an index for the file. The index contains pointers to the various blocks. To find a record, we first search the index and then use the pointer to access the file directly.

Q6 & Q9. Explain various file operations (Refer Q4). Discuss various directory structures with neat diagrams.

(Note: File operations are covered in Q4. Below is the discussion on Directory Structures.)

The directory acts as a symbol table that translates file names into their directory entries.

1. Single-Level Directory

All files are contained in the same directory.

- **Pros:** Simple to implement.
- **Cons:** Naming collision (two files cannot have the same name) and grouping problems (hard to organize files as the number increases).

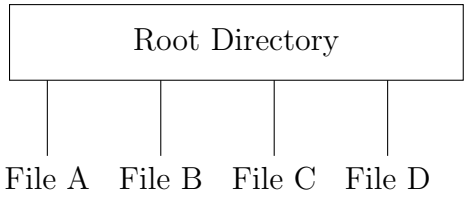

Figure 2: Single-Level Directory

2. Two-Level Directory

Each user has their own user file directory (UFD).

- **Pros:** Solves the name collision problem between different users.
- **Cons:** Isolates users; sharing files is difficult.

3. Tree-Structured Directory

This is the most common structure (used in Linux, Windows). The directory contains a set of files or subdirectories.

- **Path Names:** Every file has a unique path name (Absolute or Relative).
- **Pros:** Efficient searching, good grouping capability.

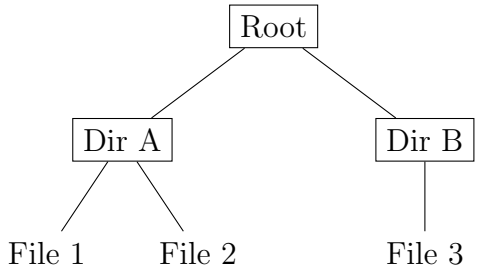

Figure 3: Tree-Structured Directory

4. Acyclic-Graph Directory

Allows directories to share subdirectories and files. The same file or directory may be in two different directories.

- Implemented via links (symbolic or hard links).
- **Issue:** Deletion is complex (dangling pointers).

Q10. Explain contiguous and linked disk space allocation methods.

1. Contiguous Allocation

Requires each file to occupy a set of contiguous blocks on the disk. Disk addresses define a linear ordering on the disk.

- **Mechanism:** The directory entry for each file indicates the address of the starting block and the length of the area allocated for this file.
- **Advantages:**
  - Excellent read performance (head movement is minimal).
  - Simple to implement (only need starting location and length).
- **Disadvantages:**
  - **External Fragmentation:** As files are allocated and deleted, free disk space is broken into little pieces.
  - File growth is difficult (if there is no space adjacent to the file).

2. Linked Allocation

Solves all problems of contiguous allocation. Each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk.

- **Mechanism:** The directory contains a pointer to the first and last blocks of the file. Each block contains a pointer to the next block.
- **Advantages:**
  - No external fragmentation.
  - File size can increase easily (as long as free blocks are available).
- **Disadvantages:**
  - **Sequential Access Only:** It is inefficient to support direct access (to find the i^{th} block, you must traverse from the start).
  - **Space Overhead:** Pointers take up space within the block.
  - **Reliability:** If a pointer is lost or damaged, the rest of the file is lost.